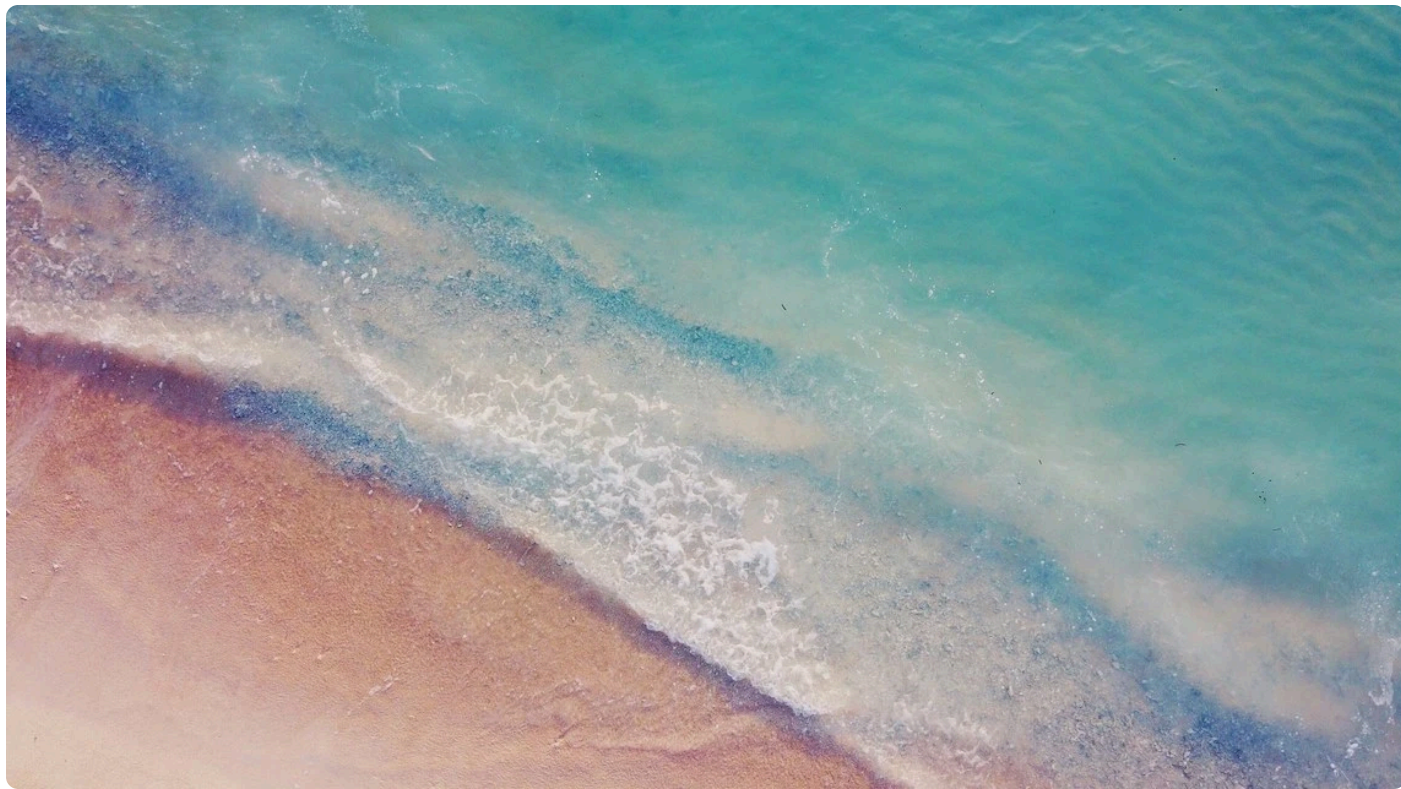


01 | 发展历程：IaC过去、现在与未来展望

潘野 · 云原生基础架构实战课



你好，我是潘野。

从今天开始，我们将会学习云原生基础架构的重要理论和发展脉络，为之后我们实践全自动化的基础架构管理打好基础。

作为课程的第一讲，我想先和你聊聊“基础设施即代码（Infrastructure as code）”。说起这个话题，你可能觉得是老生常谈，毕竟我们很容易就能在网上搜到很多相关内容。不过，这些内容往往很零散、不够系统，结合实践案例深入探讨的更是凤毛麟角。

今天，我就带你系统梳理一下基础设施即代码（Infrastructure as code）的来龙去脉。

场景模拟

让我们从一个小场景开始聊起。假如你是基础架构部门的技术总监。最近，公司要上线一个新业务。开发部门估算后，向你提出需要 300 台服务器来支持，100 台做缓存系统，100 台给

中间件使用，50 台给数据库使用，为了应对意外情况，还有 50 台做冗余。

那么接下来，从采购到业务真正上线都会经历哪些过程呢？

过去的 IaC：面向配置工具管理

在公有云普及之前，一般来说情况是这样。

1. 基础架构部门先采购 300 台服务器，数据中心工程师负责将机器推上架，加电开机。
2. 网络组同学会为这批新机器分配 Vlan、Subnet 等资源。
3. 系统运维同学根据需求分配机器，然后安装操作系统，运行配置管理工具，完成后，再交付给负责应用发布的同学。
4. 负责应用发布的同学发布应用，最终完成整个业务上线。

其中第三步是最耗费时间的，系统运维的同学不仅要安装操作系统，还要对这个 300 台的服务器做配置管理。这 300 台服务器的用途不同，所以系统配置肯定也不相同，这时就需要借助配置管理工具来帮忙。那这个配置管理工具背后是怎么运行的？它是如何保证 300 台机器能被正确管理的呢？

就拿我们比较熟悉的工具——Puppet 举例，这是一个 Client-Server 模型的配置管理工具。它的原理是这样的：每台机器上的 Puppet agent 每 30 分钟去 Puppet master 上拉取一次本机对应的配置，然后与本机上的配置做比对，发现不一致就会更改成 Puppet Master 里存储的配置。

既然 300 台机器各有各的用途，怎么使用配置管理工具来区分它们的用途，以便推送相对应的配置呢？

Puppet 内置了一个节点角色定义的功能，早期叫 ENC，Puppet 3 之后改成 Hiera，假设我们在 Puppet 节点定义文件中声明了 50 台数据库服务器，前几台数据库服务器的核心定义如下：

```
1 node 'redis01' {
2   include redis
3 }
4
5 node 'db01' {
6   include mysql
7 }
8
9 node 'db02' {
10  class {'mysql':
11    type => slave
12    default_master = "db01"
13  }
14 }
15 ...
```

结合代码可以看到，redis01 这个节点里要应用 Redis 这个模块，db01 和 db02 要应用 MySQL 这个模块，db02 里的参数表示它是 db01 的 slave 节点。Puppet agent 会根据节点定义拉取对应的配置，然后应用到主机上。

为了方便团队协作，一般运维团队会将 Puppet 的配置文件放在公司内的 git 上，通过一些代码控制流程来管理代码的变更历史，这就是我们常说的 Infrastructure as Code，简称 IaC。

基于配置管理的 IaC 方式，尽管具体选哪种工具可能有所不同，比如我们还可以选择 Saltstack 或者是 Ansible 工具，但是思路和流程大体应该差不多。

这种管理方式相比自己写脚本的传统管理方式，具有三个重要的标准。

快速：一旦节点的角色定义好了之后，Puppet agent 会自动拉取配置完成配置变更。

可靠：通过工具自动定期对比配置，防止有人手动做了更改，确保实际配置与定义配置保持一致。

可重复：无论是配置数据库服务器，还是设置冗余服务器，运维只需要定义好服务器角色，设置一次应用配置，Puppet 就能搞定所有的配置工作，省去了手工重复配置数百台机器的烦恼。

好，我们稍微总结一下什么是 IaC。IaC 是一种自动化基础设施管理的方法，通过代码描述和配置基础设施资源，实现**快速、可靠和可重复**的部署和管理过程。

现在的 IaC：面向 API 与资源管理

你有没有发现，我们前面例子里，从采购、配置 300 台服务器到业务上线的流程效率其实并不高？

首先，网络组的同学要为这批新机器分配 vlan、subnet 等网络资源，往往这个过程中还要涉及到一些开防火墙规则。其次，运维同学要等待网络组同学完成网络配置后，才能安装操作系统，并交付配置管理。

这种一环套一环的基础架构管理方式效率并不高，这几百台服务器从上架到交付使用，整个过程没有十天半个月，是不可能完成的。

这些基础设施的维护工作在业务运行里占据了很大比重，但效率低下。这显然无法跟上行业里不断变化的市场需求，拖慢了业务团队持续交付、早日上线的步伐。

为解决传统基础架构中低效工作的部分，云计算逐渐占据了基础架构中主流地位。我们不再需要自己维护一个机房，也不再需要专门采购服务器了，而是通过公有云上的 API 或者 SDK 来申请计算存储网络的资源。

那为什么公有云可以快速满足业务对基础设施资源的需求呢？在公有云上申请计算、存储和网络资源具体又是如何操作的呢？

让我们结合例子来解读，后面的命令创建了一个 EC2 虚拟机实例。这条命令背后其实是调用 AWS 的 API 来创建 EC2 虚拟机实例。

 复制代码

```
1 aws ec2 run-instances --image-id ami-xxxxxxx --count 1 --instance-type t2.micro
```

我将参数的含义整理了一张表，供你参考。

重要参数	含义
--image-id	表示这个虚拟机要用什么镜像
--count	设定了一次启动的数量
--instance-type	指的是这个虚拟机的大小
--key-name	向这个虚拟机里注入的 ssh 登陆公钥是哪一个
--security-group-ids	实例对应的安全组
--subnet-id	EC2 实例选择的 subnet



当这一条命令敲下去之后，只需要 1 分钟你就能获得一台可以立刻进行应用部署的虚拟机。其他网络资源、存储资源获取的方式也类似，对照 AWS 的文档就能轻松搞定。

可以看到在公有云上，基础架构的资源管理有了质的飞跃。

部署和上线时间方面，从原有的天级别压缩到分钟级别。

计算、网络、存储资源都可以通过 API 形式获得。

资源可以按需配置，随用随取。

在这种背景下，基础设施的配置管理就要转变成**面向云上资源管理**，也就是对计算、网络、存储做全方位的管理，而不是再面向机器配置管理。然而，基于 Puppet 这类配置管理工具的 IaC 方式显然无法匹配新的情况，因为它们的能力仅限于主机上的配置管理。

那么面向资源管理具体怎么实现呢？

我们主要会通过命令式和声明式两种方式来管理资源。我继续沿用这个业务上线的例子，来帮你理解命令式与声明式。

此时需求会变成这样：

应用需要 30 台虚拟机部署中间件，30 台虚拟机部署缓存系统，这两类机器启用云上自动扩展功能，根据 CPU 使用率对机器数量进行扩容。

前端需要一个负载均衡器，对外暴露的端口是 8082。

10 个云数据库实例用来存储用户信息，用户的图片数据存储在对象存储中，对象存储的大小为 1T。

这时运维同学就可以根据上述需求，将 AWS 的命令组合成一个 shell 的脚本。脚本中会组合地创建 EC2 实例、RDS 数据库实例、创建 ELB 还有创建 S3 的命令。然后执行这个脚本即可轻松获取到相对应的资源，这种管理方式叫**命令式方法**。

命令式方法的问题在于配置程度低，一旦应用多起来，就会带来更多的手工工作。

于是，一种新的资源管理方式就出现了。AWS 为了帮助用户简化工作，快速复制基础设施，推出了 CloudFormation 这个功能。CloudFormation 使用结构化的文本，通过模板方式组合来创建一组资源。这些资源能一起合作，共同创建一个应用程序或解决方案。

这种方式叫做**声明式方法**。声明式方法定义了系统的预期状态，包括所需的资源以及它们应具有的属性，声明式方法的 IaC 工具执行时可能会中断或出错，但是这类工具可以不断重试，直到最终的实际状态与预期状态一致。

我们来看一个 CloudFormation 的例子，直观比较一下声明式方法和之前的命令式方法有什么不同。

 复制代码

```
1 Resources:
2   Ec2Instance:
3     Type: 'AWS::EC2::Instance'
4     Properties:
5       SecurityGroups:
6         - !Ref InstanceSecurityGroup
7       KeyName: mykey
8       ImageId: 'ami-xxxx'
9   InstanceSecurityGroup:
10    Type: 'AWS::EC2::SecurityGroup'
```

```
11 Properties:
12   GroupDescription: Enable SSH access via port 22
13   SecurityGroupIngress:
14     - IpProtocol: tcp
15       FromPort: 22
16       ToPort: 22
17       CidrIp: 0.0.0.0/0
```

在这个配置中，我们定义了这个 EC2 的镜像、安全组、ssh 登陆公钥等信息，将命令转化成了声明式的配置。之后再结合模版的方式，就能达到 IaC 管理**快速、可靠、可重复的标准**。

但 CloudFormation 是 AWS 的产品，只支持 AWS 的 API。如果我们选择其他的云厂商或者是自己搭建私有云时，又该怎么办呢？有没有什么工具可以支持所有的 API？

于是，Terraform 这种多云基础设施编排工具就应运而生了！和 CloudFormation 一样，Terraform 也是声明式方法，但是它没有使用结构化文本，而是 Terraform 自己的语法，这里我贴一个小样例。

 复制代码

```
1 module "ec2_instance" {
2   source = "terraform-aws-modules/ec2-instance/aws"
3
4   name = "single-instance"
5
6   instance_type      = "t2.micro"
7   key_name           = "user1"
8   monitoring         = true
9   vpc_security_group_ids = ["sg-12345678"]
10  subnet_id          = "subnet-eddczz4"
11
12  tags = {
13    Terraform = "true"
14    Environment = "dev"
15  }
16 }
```

可以看到，这个例子中导入了 terraform-aws-modules，因此能够支持 Google Cloud，Azure Cloud 等其他云的模块。下一讲我还会跟你继续探讨 Terraform 的原理和应用，这里

你先对它有个印象就行。

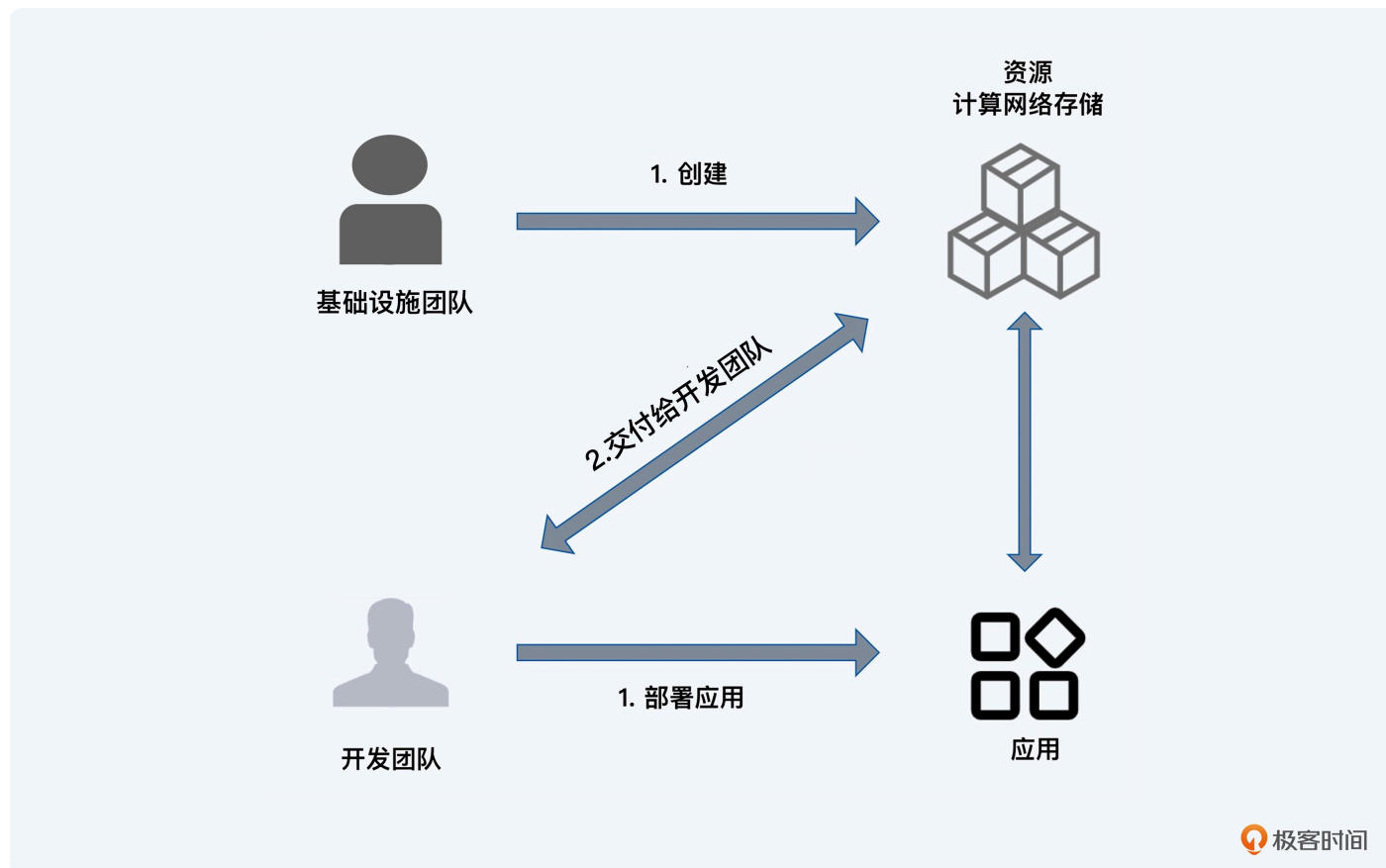
未来的 IaC：面向应用管理

早期 IaC 是为系统管理员服务，通过 Puppet 这样的工具帮助系统管理员自动化配置 server；现在的 IaC 是面向资源，通过 Terraform 这样的工具将计算网络存储的管理整合在一起。这两种管理方式的共性是，先由应用申请资源，再由我们为应用准备资源。

但是还有没有更快更好的方法呢？

我们再回顾下前面模拟场景里的需求，开发团队上线应用程序之前，需要等待基础设施团队通过云的 API 准备资源。那么开发团队为什么不能直接调用云的 API 呢？

但是，哪怕开发团队自学了 Terraform 的文档，搞定了冗长的资源配置管理操作，再应用到云上，整个过程其实是开发团队被动完成了基础设施团队的工作，本质上并没有让效率变得更高。

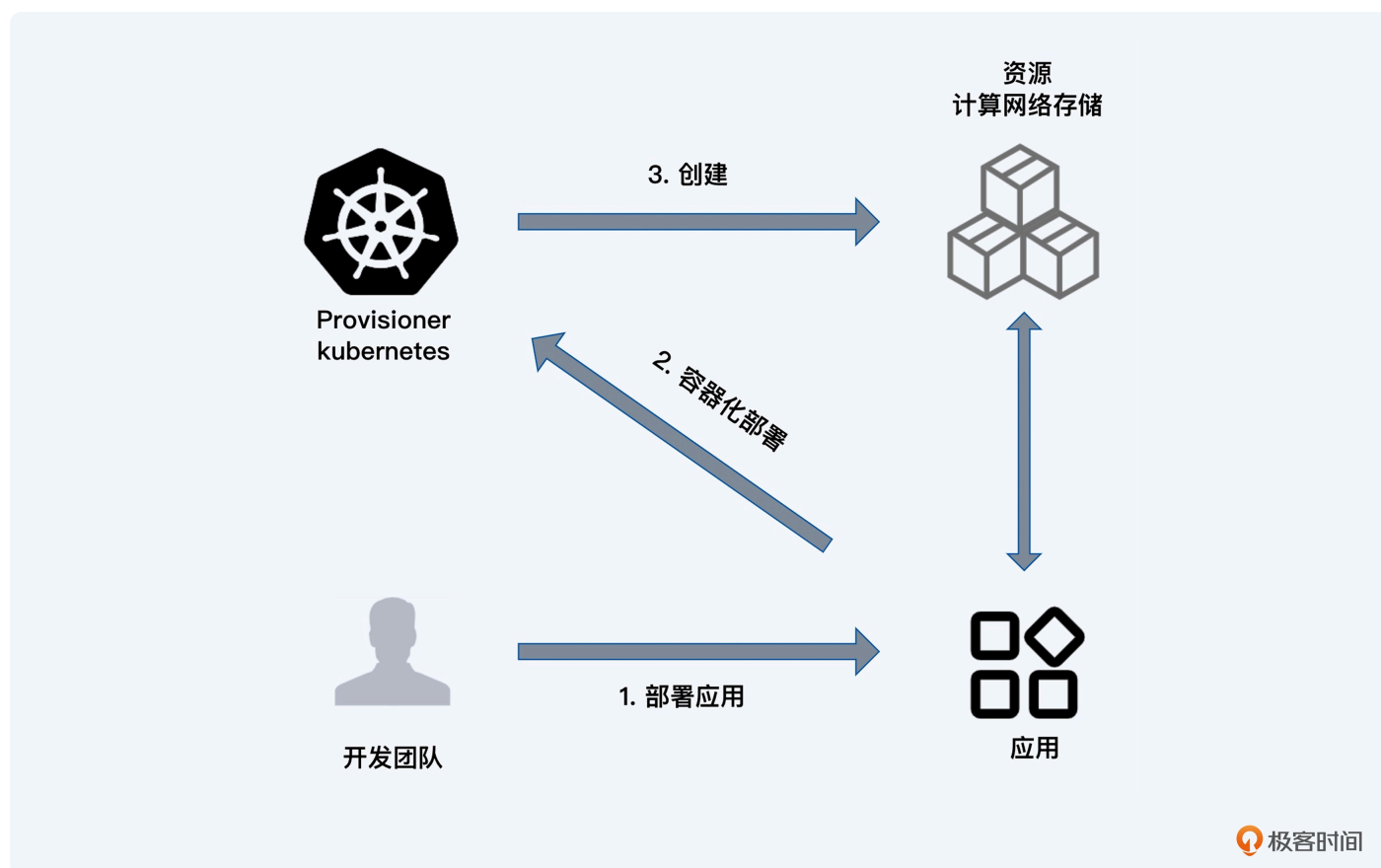


所以这种交付方式依赖的其实是某个人或者某个岗位，只有准备好资源，才能分配给应用资源使用。

一旦有很多应用程序需要调用资源，这种“按需下单，等候分配”的过程就会更加漫长、复杂。那么能不能把准备资源这个过程，交由应用程序本身来触发，通过自动化程序来管理应用程序的整个生命周期呢？

这不就是 Kubernetes 最重要的特性吗？Kubernetes 实际上将基础设施转化成了它的配置对象，让业务在大部分场景中不需要去考虑底层基础架构的形态。

这样一来，开发团队在完成程序开发之后，不需要找到系统管理员或者云管理员，只要在 Kubernetes 配置文件里，声明所需要 CPU、内存和磁盘的大小以及对外暴露服务需要的网关等信息，通过 Kubernetes API 便可调用各类资源。当应用生命周期结束的时候，再通过 Kubernetes 配置文件删除应用并释放资源，做到资源的有借有还。



今天 Kubernetes 已经是事实上的云计算的标准接口。Kubernetes 通过对计算、网络、存储的高度抽象，屏蔽了云上和云下的差异。我们在云上和云下机房里分别启动 Kubernetes 的集

群，应用程序几乎不用更改，就可以很快地从云下迁移到云上，或者从云上迁移到云下。

在我看来，在云原生技术的帮助下，未来的基础架构管理中应用与基础设施之间将不再脱节。Kubernetes 一方面会为应用提供接口，快速获得相应的计算、网络与存储资源，另一方面又能对接云厂商的 API，实现根据应用需求动态扩展资源的能力。

所以，未来 Kubernetes 的集群管理将会变成 IaC 的主体。而对于基础设施团队，可以把更多的关注点更多放在集群组件管理、集群监控、成本分析、资源优化等进阶工作上。

总结

今天我和你分享了我对基础设施即代码的经验和理解。

我们用一句话来总结一下 IaC 的概念，IaC 是一种自动化基础设施管理的方法，通过代码描述和配置基础设施资源，实现**快速、可靠和可重复**的部署和管理过程。



IaC 的发展过程可以分为三个阶段。最初，我们主要关注的是面向主机的配置管理，这包括安装和配置服务器、数据库等基础设施组件。随着云计算的发展，我们的关注点转向了面向云资源的管理，这包括创建和管理虚拟机、存储、网络等云资源。在未来，我们预计将进一步前进，关注面向应用资源的管理，这将包括部署和管理微服务、容器、函数等应用级别的资源。

下一讲，我会带你梳理现在流行的 IaC 的工具，帮你选择适合自己业务的 IaC 工具。

思考题

请你对照 IaC 方式的三大标准，看看自己公司里现在所实施的 IaC 存在哪些不足？

欢迎你在留言区和我交流互动。如果今天的内容对你有启发，也推荐你分享给身边更多朋友。

AI智能总结

本文主要介绍了基础设施即代码（IaC）的发展历程和管理方式。首先通过一个场景模拟引出了IaC的发展脉络，然后详细介绍了过去的IaC管理方式，即面向配置工具管理的情况。随后，文章介绍了现在的IaC，即面向API与资源管理的方式，以及展望了未来的IaC，即面向应用管理。整体来看，本文通过具体案例和详细解释，系统梳理了IaC的发展历程和管理方式，为读者打下了扎实的基础。IaC是一种自动化基础设施管理的方法，通过代码描述和配置基础设施资源，实现快速、可靠和可重复的部署和管理过程。文章还展望了未来的IaC，即面向应用管理，通过Kubernetes将基础设施转化成配置对象，实现了资源的有借有还，将集群管理作为IaC的主体。整体来看，本文通过具体案例和详细解释，系统梳理了IaC的发展历程和管理方式，为读者打下了扎实的基础。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (3)

最新 精选



yayiyaya

2024-03-26 来自浙江

老师，我有些地方不明白，声明需要CPU、内存和磁盘。可以通过定义的resource 资源和storageclass来解决；如果是节点的资源不足，也可以通过云平台k8s管理，动态的扩缩容节点。但是，何时进行扩缩容？是通过什么做判断的？某些服务申请的声明资源比实际使用要大的多，这些又该怎么办呢？

作者回复: 同学你好，你提的问题非常好，这也是我们模块三中关于资源动态扩缩容中重要的内容。在模块三中，我们会详细了解，应用平面的动态扩缩容与集群层面的动态扩缩容是如何联动的，敬请期待。

共 3 条评论 >



1



xueerfei007

2024-04-22 来自浙江

但是对于云服务提供者来说，给外部提供k8s这样的api，方便用户创建使用资源，但是他们自己的工作依然很复杂，还是要自己处理物理集群

作者回复: 在基础设施的管理中，一直都一个问题叫“先有鸡还是先有蛋”。那么用户通过k8s的API去获取了资源，那么管理员就要为这个API去做很多的事情，这也是这门课程想要去达到的一个目标。



kaizen

2024-03-25 来自北京

应用所需要的DB,消息队列等，应该还得靠terraform来拉起来吧

作者回复: Hi，同学你好

其实有很多方式可以将中间件，DB这些资源拉起来，比如你说的用terraform，也可以用容器化的方式部署进kubernetes，也可以用一些Operator来拉起这些资源。不同的拉起方式，有不同的管理方法，在模块2中我们会讲解怎么去管理这些资源。

共 2 条评论 >

